11.11 Secrets Scanning

Topic 11 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Secrets scanning automatically searches code, Git history, configuration, container images and CI logs for credentials that should never be there: cloud access keys, database URLs with passwords, API tokens, private keys, webhook URLs. Scanners use two techniques together:

- **Pattern rules (regular expressions)** for well-known formats, such as AWS access key ids starting with `AKIA`, GitHub tokens starting with `ghp_`, and `-----BEGIN PRIVATE KEY-----` blocks.
- **Entropy and context heuristics** for random-looking strings assigned to variables named `password`, `secret` or `token`.

Some tools add **verification**: **TruffleHog** can call the provider's API to check whether a detected key is still live, so you can focus on real exposure.

The tools in this topic: **gitleaks** (fast, open source, easy to run as a pre-commit hook and in CI), **TruffleHog** (deep history scanning and verification), and **GitHub secret scanning with push protection** (built into GitHub; blocks a push that contains a recognised secret before it lands in the repository).

2 Why it matters in DevOps

- **A secret in Git is a secret forever.** Every clone, fork and CI cache contains it; deleting the file in a later commit does nothing to history.
- **Attackers scan too.** Automated bots watch public GitHub activity and try newly exposed cloud keys within minutes; the realistic assumption is that a leaked key is compromised, not "probably unnoticed".
- **Blast radius is set by the key's permissions.** An admin key leaked from a dev laptop can mean a crypto-mining bill, a data breach or deleted infrastructure (least-privilege IAM §11.2).

- **Prevention is cheap, cleanup is expensive.** Blocking at commit or push time takes seconds; rotating a credential used by twelve services takes a day.

3 Real-world system example

Uber (2016): according to the US Federal Trade Commission's and Department of Justice's public documents, attackers obtained AWS credentials from a private GitHub repository used by Uber engineers and used them to access data stored in Amazon S3. It is a standard example of why private repositories are not secret stores.

Codecov (2021): attackers modified Codecov's Bash Uploader script, which customers ran in CI, so that it sent the environment variables of those CI jobs (including any tokens and keys) to an attacker-controlled server. Customers were advised to rotate every credential that had been exposed to those jobs. The lesson is twofold: CI environments are full of secrets, and a leaked secret must be *rotated*, not just hidden.

CircleCI (January 2023): after a security incident, CircleCI told customers to rotate all secrets stored in the platform.

GitHub documents secret scanning for hundreds of token types across a partner programme in which providers (cloud vendors, SaaS companies) are told when their token format appears in a public repository so they can revoke or notify. Push protection extends that to blocking the push in the first place. (AWS, for example, is documented to respond to exposed keys by applying a quarantine policy and emailing the account owner, which limits damage but is not a substitute for rotating the key.)

4 Diagram

Defence in depth: four chances to catch a secret, from cheapest to most expensive:

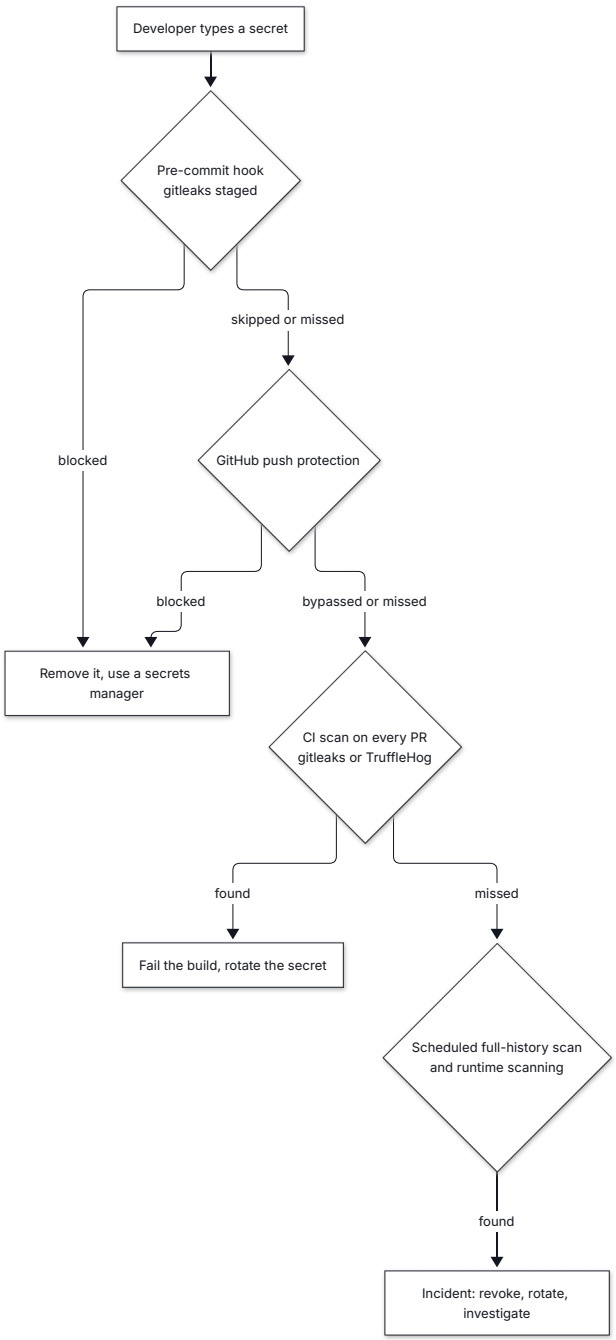


Figure 11.11-1

The incident flow when a secret has leaked. Order matters: revoke first, clean later.

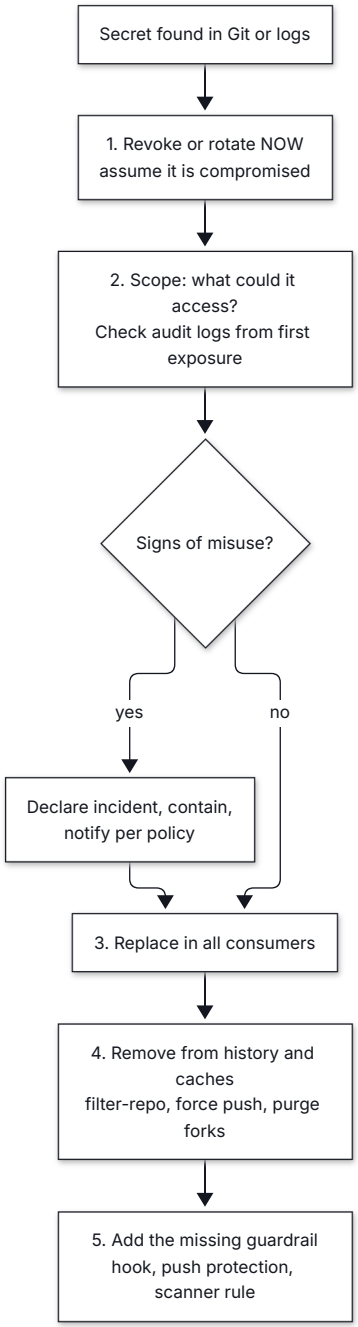


Figure 11.11-2

5 How it works

Where scanners look

Surface	How to scan
Working tree and staged changes	pre-commit hook (`gitleaks git --pre-commit --staged`)
Full Git history	`gitleaks git` , `trufflehog git file://.` (including deleted files and old branches)
Pull-request diffs	CI step on every PR, scanning only the new commits
Container images and build contexts	`trivy image --scanners secret` , `trivy fs --scanners secret` (Container Image Scanning §11.9)
CI logs and artifacts	Masking in the CI platform; scanners on uploaded artifacts
Hosted repos	GitHub secret scanning and push protection; GitLab secret detection

Tool comparison

Tool	Strength	Notes
gitleaks	Fast, simple TOML config, great for hooks and CI	Custom rules, allowlists, baselines (`--baseline-path`) for existing findings
TruffleHog	Detects many credential types; `--only-verified` checks whether they still work	Slower on large histories but low false positives when verifying
GitHub secret scanning + push protection	No setup for supported token types; blocks pushes	Free for public repositories; for private repositories it is part of GitHub's paid security offering. Check the current plan options
detect-secrets (Yelp)	Baseline-driven workflow for large legacy repos	Good for gradually cleaning up

False positives. Test fixtures, documentation keys (`AKIAIOSFODNN7EXAMPLE`) and hashes trigger rules. Handle them with a *narrow* allowlist (specific path or fingerprint) and a comment, never by disabling a rule globally. A baseline file lets you accept existing findings while failing on new ones.

What "remove from history" does and does not do. Rewriting history with `git filter-repo` (or BFG Repo-Cleaner) plus a force push removes the secret from the branch tips, but clones, forks, pull-request refs, CI caches and anyone who already fetched it still hold it. Therefore rewriting is **hygiene**, and **rotation is the fix**.

6 Step-by-step guide

Prerequisites: Git, Docker (to run scanners without installing them). Everything happens in a throwaway local repo with a **fake** token. No real secret is used and nothing is pushed anywhere. Output blocks marked "similar to" will differ by tool version.

1. **Create a repo and commit a fake secret.**

```bash
mkdir -p ~/secrets-lab && cd ~/secrets-lab && git init -q
git config user.email lab@example.com && git config user.name "Lab User"
cat > config.js <<'EOF'
// fake token generated for this lab, not a real credential
module.exports = { shopApiToken: "shop_live_a1b2c3d4e5f6g7h8i9j0k1l2" };
EOF
git add config.js && git commit -qm "add config"
echo 'module.exports = { shopApiToken: process.env.SHOP_API_TOKEN };' > config.js
git commit -qam "remove token from config (but it is still in history!)"
git log --oneline
```

   Two commits. The file now has no secret, but `git show HEAD~1:config.js` still prints it. That is the trap.

2. **Write a custom gitleaks rule** (also shows how teams detect their own token formats).

```bash
cat > .gitleaks.toml <<'EOF'
title = "shop-api gitleaks config"

[extend]
useDefault = true            # keep all built-in rules

[[rules]]
id = "shop-api-token"
description = "shop-api live API token"
regex = '''shop_live_[a-z0-9]{24}'''
keywords = ["shop_live_"]
EOF
```

3. **Scan the working tree, then the history.**

```bash
docker run --rm -v "$PWD":/repo zricethezav/gitleaks:latest dir /repo -c
  /repo/.gitleaks.toml --redact
docker run --rm -v "$PWD":/repo zricethezav/gitleaks:latest git /repo -c
  /repo/.gitleaks.toml --redact -v
```

The first command (current files) reports no leaks. The second, which walks commits, reports output similar to:

```
Finding:     shopApiToken: "REDACTED"
Secret:      REDACTED
RuleID:      shop-api-token
File:        config.js
Line:        2
Commit:      <sha of the first commit>
Fingerprint: <sha>:config.js:shop-api-token:2
... WRN leaks found: 1
```

The exit code is 1 when leaks are found. If your gitleaks version uses the older subcommands, use `gitleaks detect --source /repo` in place of `git`, and `--no-git` in place of `dir`.

4. **Run TruffleHog on the same history.**

```bash
docker run --rm -v "$PWD":/repo trufflesecurity/trufflehog:latest git file:///repo --
  no-update
```

Without custom detectors it will not know your internal token format, so expect `finding 0` for this fake: a useful reminder that scanners know *formats*. Its strength shows with real provider keys, where `--only-verified` shows only live credentials.

5. **Add a pre-commit hook** so the secret never lands in the first place. With the pre-commit `(pre-commit.com)` framework, create `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: <LATEST_RELEASE_TAG>      # pin to a release tag, e.g. v8.x.y, and update via
  Dependabot or pre-commit autoupdate
    hooks:
      - id: gitleaks
```

```bash
pip install pre-commit && pre-commit install
echo 'const k = "shop_live_zzzzzzzzzzzzzzzzzzzzzzzz";' > leak.js && git add leak.js
pre-commit run --all-files        # should fail with a gitleaks finding
git reset -q leak.js && rm leak.js
```

(Adjust `.gitleaks.toml` discovery as needed; by default the hook reads `.gitleaks.toml` at the repo root.)

6. **Add a CI scan** (`.github/workflows/secrets.yml`):

```yaml
name: secrets-scan
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "41 2 * * 1"          # weekly full-history rescan
permissions:
  contents: read
jobs:
  gitleaks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0           # full history, otherwise only the tip is scanned
      - name: Gitleaks (pinned container image)
        run: |
          docker run --rm -v "$PWD":/repo zricethezav/gitleaks:<PINNED_VERSION> \
            git /repo --redact --exit-code 1
```

`fetch-depth: 0` matters: a shallow clone hides old leaks. Use a pinned scanner version, and the repository's `.gitleaks.toml` if present (add `-c /repo/.gitleaks.toml`). On GitHub, also enable **Settings > Advanced Security (Code security) > Secret scanning** and **Push protection** where your plan allows.

7. **Practise the response: "the token leaked" drill (on the fake token).** Do the steps in order and note the time each takes:

```bash
# 1. REVOKE/ROTATE FIRST. In real life: disable the key in the provider's console and
  issue a new one.
#    (Here: write down "shop_live_a1b2... revoked" in your incident notes.)
# 2. Find out where it spread
git log --all -S'shop_live_' --oneline            # every commit that added or removed
  the string
git grep -n 'shop_live_' $(git rev-list --all) | head
# 3. Rewrite history to remove it from every commit (install git-filter-repo first)
pip install git-filter-repo
printf 'shop_live_a1b2c3d4e5f6g7h8i9j0k1l2==>REMOVED_SECRET\n' > /tmp/replace.txt
git filter-repo --replace-text /tmp/replace.txt --force
git log --all -S'shop_live_' --oneline            # should print nothing
```

(`git filter-repo` also removes the `origin` remote as a safety measure; re-add it before pushing.) In a real repository you would then force-push all branches and tags, ask collaborators to re-clone (not merge), contact the host to purge cached views of old commits and forks, and, again, **the key stays revoked regardless**.

8. **Clean up.** `cd ~ && rm -rf ~/secrets-lab /tmp/replace.txt`

7 Real-world scenario

Illustrative, not a documented incident. At 17:40 on a Friday a developer pushes a debugging branch to a public fork containing a `.env.production` file with the `shop-api` payment provider key. GitHub's secret scanning notifies the provider, which emails the account owner; at the same moment a CI full-history scan opens an alert. The on-call engineer follows the runbook.

Revoke the key at the payment provider (18:05), then generate a new one and deploy it to Vault so the service reads it at runtime (18:20). Review the provider's API logs from 17:40 onwards: two unfamiliar requests from an unknown IP, both read-only. The team documents it as a security event, removes the file from the fork's history, and adds `.env*` to `.gitignore`, a pre-commit hook and push protection.

The post-incident review finds that the root cause was a long-lived key shared by all environments, so the new setup issues per-environment, restricted keys.

Lesson: the sequence was revoke, scope, replace, clean, prevent. Rewriting the Git history was the last and least important step. Detection speed set the exposure window (minutes, not months).

8 Common mistakes and best practices

Don't

- Delete the secret in a new commit and consider it solved.
- Clean history first and rotate "when there is time".
- Assume a private repository or a deleted fork is safe.
- Disable the scanner (or `--no-verify` the hook) because of a false positive.
- Put secrets in CI logs, build arguments or image layers; scanners find these too.
- Share one all-powerful key across environments so that a single leak is total.

Do

- Treat every exposed secret as compromised: revoke or rotate immediately, then investigate with audit logs.
- Layer controls: pre-commit hook, push protection, CI scan on PRs, scheduled full-history scans, image scans.
- Use fetch-depth 0 for history scans; scan new commits on PRs for speed and the whole history on a schedule.
- Keep allowlists narrow and commented; use a baseline for legacy findings and burn it down.
- Move real secrets to a manager and give workloads short-lived credentials (Secrets Management §11.4, CI secrets §5.5).
- Rehearse the leak runbook so the first real one is routine.

9 Tasks

- [ ] **T1 — Find a secret in history** *(≈30 min, beginner)*
  - **Goal:** prove that deleting a file does not delete a secret.
  - **Steps:**
    1. Follow steps 1-3 of the guide.
    2. Show the secret is absent from `HEAD` but present with `git show HEAD~1:config.js`.
    3. Show gitleaks detects it only in history mode.
  - **Done when:** `gitleaks dir` reports no leaks and `gitleaks git` reports one, with the commit sha.
- [ ] **T2 — Block it before commit** *(≈30 min, intermediate)*
  - **Goal:** shift detection to the developer's machine.
  - **Steps:**
    1. Install the pre-commit hook (step 5).
    2. Try to commit a file containing a fake `shop_live_` token.
    3. Read the failure message, remove the token, and commit successfully.
  - **Done when:** the first commit attempt is rejected by the hook and the second succeeds.
- [ ] **T3 — Tune out a false positive correctly** *(≈30 min, intermediate)*
  - **Goal:** keep the scanner trusted.
  - **Steps:**
    1. Add a test fixture containing the documented example key `AKIAIOSFODNN7EXAMPLE` (or any rule-matching dummy value) under `test/fixtures/`.

2. Add a path-scoped allowlist in `.gitleaks.toml` (`[allowlist]` or `[[allowlists]]` with `paths`, depending on your gitleaks version), with a comment on why.
3. Confirm the fixture is ignored while the same pattern in `src/` is still reported.

- **Done when:** a scan reports a finding for `src/` and none for `test/fixtures/`.

- [ ] **T4 — Put it in CI and fail a PR** *(≈45 min, intermediate)*
  - **Goal:** enforce scanning on every pull request.
  - **Steps:**
    1. Add the workflow from step 6 to a test repository.
    2. Open a PR that adds a fake token. Observe the red check.
    3. Push a fix commit that removes the token, and note that the check still sees the token in the PR's history (the scan covers commits). Rewrite the branch (`git rebase -i` / squash and force-push to the PR branch) so the commit disappears, then confirm green.
  - **Done when:** the PR goes red then green and you can explain why the fix commit alone was not enough.

- [ ] **T5 — Run the leak drill end to end** *(≈60 min, advanced)*
  - **Goal:** execute the incident runbook calmly.
  - **Steps:**
    1. Create a free low-risk credential you control (for example a throwaway GitHub personal access token with no scopes, or a fake provider token in a local mock) and "leak" it into your lab repo.
    2. Follow the sequence: revoke, scope, replace, clean history with `git filter-repo`, add prevention.
    3. Write a five-line incident note with timestamps.
  - **Done when:** the credential is revoked (verify a request using it fails), history no longer contains it, and the note shows the order of actions.

- [ ] **T6 — Prove it: write the team runbook** *(≈30 min, advanced)*
  - **Goal:** make the response repeatable by someone else.
  - **Steps:**
    1. Without notes, write a one-page "Secret leaked" runbook for `shop-api` that names who revokes each credential type (database, cloud, payment, JWT signing key) and how to check for misuse.
    2. Include what *not* to do first.

- **Done when:** a teammate who did not write it can follow it for a leaked AWS key and a leaked database password.

10 Self-check questions

1. A developer commits a key and removes it in the next commit. Is the repo safe? What do you do?
2. Why should rotation come before history rewriting?
3. What do `fetch-depth: 0` and `--only-verified` each change?
4. Why can pattern-based scanners miss a secret, and how do you cover that gap?
5. Why is a private repository not a secrets manager?

ANSWERS

1. No: the secret is still in history and in every clone. Revoke and rotate it, check audit logs for misuse, then optionally rewrite history and force-push as hygiene.
2. Rewriting does not reach existing clones, forks or caches, whereas revoking makes the leaked value useless wherever it is; the attacker's window starts at exposure, not at your cleanup.
3. `fetch-depth: 0` makes CI clone full history so older commits are scanned; `--only-verified` makes TruffleHog report only secrets it confirmed are still valid with the provider.
4. Rules know formats and heuristics; custom or unusual secrets slip past. Add custom rules, entropy detection, and avoid secrets in code altogether by using a manager.
5. Access to a repo is broad (every engineer, contractor, CI integration and clone), has no per-secret audit trail or rotation, and repositories get made public, forked or leaked.

11 Further reading

- gitleaks documentation (the repository README covers configuration, baselines and hooks) and the TruffleHog documentation.
- GitHub Docs: *About secret scanning* and *About push protection*.
- `git-filter-repo` documentation and the BFG Repo-Cleaner for history rewriting.
- OWASP Cheat Sheet Series: *Secrets Management*; the FTC/DOJ public documents on the Uber 2016 breach for a real-case read.